Tecnicatura Superior en Desarrollo de Software

Módulo 10 — `Blazor Hybrid`: la pantalla de alta de clientes

Actividad Práctica — CU-CLI-001: Registrar Cliente | Asignatura: Programación II | Instituto de Educación Superior N.º 9007 Dr. Salvador Calafat | Ciclo Lectivo 2026

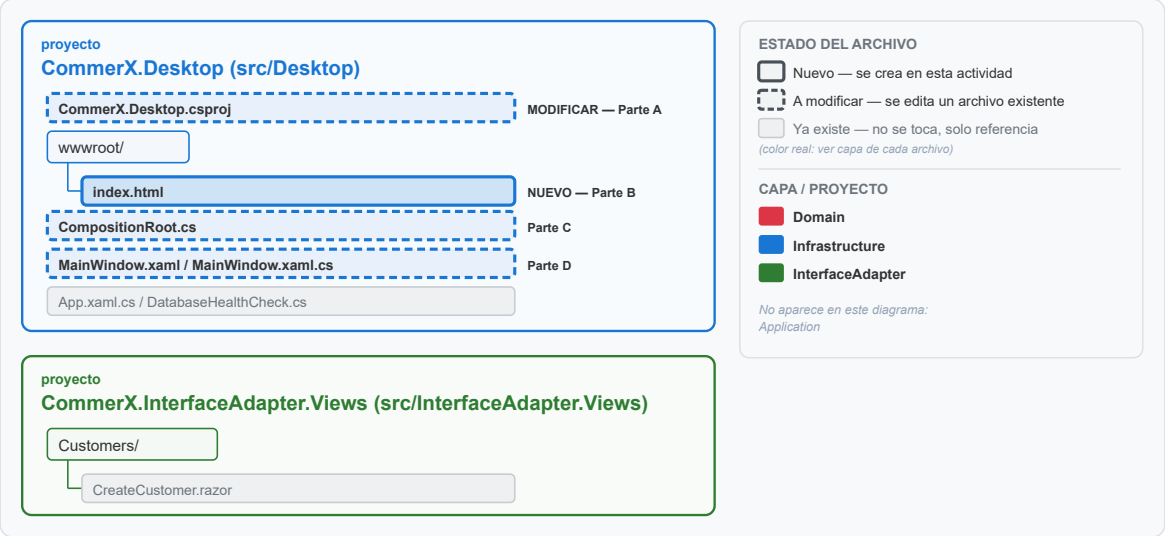
Contexto de la actividad

Este es el último paso del caso de uso *Registrar Cliente*. En el Módulo 7 escribiste la View (`CreateCustomer.razor`), en el Módulo 8 la persistencia y en el Módulo 9 un host que arma todas las dependencias y comprueba la base de datos. Pero la ventana del host todavía muestra solo un texto: la View nunca se mostró en pantalla.

En esta actividad conectás las dos mitades. Vas a agregar a la ventana de WPF un control llamado **BlazorWebView**, que sabe mostrar componentes Razor dentro de una aplicación de escritorio. Al terminar, vas a poder completar el formulario, tocar *Guardar* y ver el cliente guardado en SQL Server. No escribís ninguna regla nueva ni tocás ninguna capa interior: **todo lo que hacés es de la capa más externa**, el host.

> **Punto de partida:** la solución tal como quedó al terminar el Módulo 9 (el host `CommerX.Desktop` funcionando, con la base de datos `CommerX` creada). Además hace falta el **runtime de WebView2**, que ya viene instalado con Windows 10 y 11 actualizados (lo trae Microsoft Edge). Si el programa se abre pero el formulario aparece en blanco, es lo primero que hay que revisar.

Archivos involucrados



Cuatro archivos del host cambian y uno se crea. El archivo en gris, la View del Módulo 7, **no se toca**: se muestra tal cual la escribiste. Que la View no cambie es justamente la prueba de que el diseño en capas funciona.

Parte A — Preparar el proyecto del host: SDK, referencia y paquete

Ubicación

`src/Desktop/CommerX.Desktop.csproj` (archivo existente, se modifica)

Descripción

Blazor Hybrid es la forma de mostrar componentes Razor (los mismos que se usan en una página web) dentro de una aplicación de escritorio. El componente no se muestra con controles de WPF: se dibuja dentro de un navegador embebido, **WebView2**, que es el motor de Microsoft Edge. Pero no hay servidor web ni internet de por medio: el código C# corre dentro del mismo programa y habla con la página directamente. Para lograrlo, el host necesita tres cosas, que van todas en su `.csproj`: el SDK correcto, la referencia a las Views y el paquete del control.

Esto es lo que tiene hoy el archivo (como quedó en el Módulo 9):

```
<Project Sdk="Microsoft.NET.Sdk">

  <ItemGroup>
    <ProjectReference Include="..\InterfaceAdapter\CommerX.InterfaceAdapter.csproj" />
    <ProjectReference Include="..\Infrastructure\CommerX.Infrastructure.csproj" />
  </ItemGroup>

  <ItemGroup>
    <PackageReference Include="Microsoft.Extensions.Hosting" Version="9.0.20" />
  </ItemGroup>

  <ItemGroup>
    <!-- appsettings.json se copia junto al ejecutable para que el host lo encuentre al arrancar -->
    <None Update="appsettings.json">
      <CopyToOutputDirectory>PreserveNewest</CopyToOutputDirectory>
    </None>
  </ItemGroup>

  <PropertyGroup>
    <OutputType>WinExe</OutputType>
    <TargetFramework>net9.0-windows</TargetFramework>
    <Nullable>enable</Nullable>
    <ImplicitUsings>enable</ImplicitUsings>
    <UseWPF>true</UseWPF>
  </PropertyGroup>

</Project>
```

Paso 1 — Cambiar el SDK a Razor

La primera línea dice con qué conjunto de herramientas se compila el proyecto. Hoy es `Microsoft.NET.Sdk`. Para que el host pueda publicar los archivos que necesita el `BlazorWebView` (por ejemplo, el script `blazor.webview.js`), tiene que ser `Microsoft.NET.Sdk.Razor`: es el mismo SDK de las bibliotecas de Razor del Módulo 7.

Paso 2 — Referenciar el proyecto de Views

El componente `CreateCustomer` vive en `CommerX.InterfaceAdapter.Views`. El host tiene que conocerlo para poder mostrarlo:

```
dotnet add src/Desktop/CommerX.Desktop.csproj reference src/InterfaceAdapter.Views/CommerX.InterfaceAdapter.Views.csproj
```

En Visual Studio: clic derecho sobre **Dependencias** del proyecto `CommerX.Desktop` → **Agregar referencia de proyecto…** → marcá `CommerX.InterfaceAdapter.Views`.

Paso 3 — Instalar el paquete del BlazorWebView

`Microsoft.AspNetCore.Components.WebView.Wpf` es el paquete NuGet que trae el control `BlazorWebView` para WPF. Como siempre, versión exacta de la rama de la solución (9.x):

```
dotnet add src/Desktop/CommerX.Desktop.csproj package Microsoft.AspNetCore.Components.WebView.Wpf --version 9.0.120
```

En Visual Studio: clic derecho sobre el proyecto → **Administrar paquetes NuGet…** → pestaña **Examinar** → `Microsoft.AspNetCore.Components.WebView.Wpf` → versión `9.0.120` → **Instalar**.

Con los tres pasos hechos, el archivo completo queda así:

```
<Project Sdk="Microsoft.NET.Sdk.Razor">

  <ItemGroup>
    <ProjectReference Include="..\InterfaceAdapter\CommerX.InterfaceAdapter.csproj" />
    <ProjectReference Include="..\Infrastructure\CommerX.Infrastructure.csproj" />
    <ProjectReference Include="..\InterfaceAdapter.Views\CommerX.InterfaceAdapter.Views.csproj" />
  </ItemGroup>

  <ItemGroup>
    <PackageReference Include="Microsoft.Extensions.Hosting" Version="9.0.20" />
    <PackageReference Include="Microsoft.AspNetCore.Components.WebView.Wpf" Version="9.0.120" />
  </ItemGroup>

  <ItemGroup>
    <!-- appsettings.json se copia junto al ejecutable para que el host lo encuentre al arrancar -->
    <None Update="appsettings.json">
      <CopyToOutputDirectory>PreserveNewest</CopyToOutputDirectory>
    </None>
```

```
19    </ItemGroup>
20
21    <PropertyGroup>
22      <OutputType>WinExe</OutputType>
23      <TargetFramework>net9.0-windows</TargetFramework>
24      <Nullable>enable</Nullable>
25      <ImplicitUsings>enable</ImplicitUsings>
26      <UseWPF>true</UseWPF>
27    </PropertyGroup>
28
29  </Project>
```

Cómo leerlo:
Línea 1: el SDK pasa a `Microsoft.NET.Sdk.Razor` (paso 1).

Línea 6: la referencia al proyecto de Views (paso 2).

Línea 11: el paquete del `BlazorWebView`, con versión exacta (paso 3).

Fijate que **no hace falta** ninguna línea para la carpeta `wwwroot` de la Parte B: el SDK Razor la publica solo.

> **Comprobá:** ejecutá `dotnet build src/Desktop/CommerX.Desktop.csproj`. Tiene que terminar con 0 errores y 0 advertencias. (Todavía no existe la carpeta `wwwroot`, y eso no impide compilar.)

Parte B — Crear la página `wwwroot/index.html`

Ubicación

`src/Desktop/wwwroot/index.html` (crear la carpeta `wwwroot` y el archivo)

Descripción

El navegador embebido necesita una página HTML donde "pegar" el componente. Es una página casi vacía: tiene un lugar reservado y un script. Todo lo demás lo arma Blazor. Creá la carpeta y el archivo con este contenido:

```
1  <!DOCTYPE html>
```

```
2  <html lang="es">
3
4  <head>
5      <meta charset="utf-8" />
6      <title>CommerX</title>
7      <base href="/" />
8      <style>
9          body { font-family: "Segoe UI", sans-serif; margin: 0; padding: 16px 24p
   x; }
10         label { display: block; margin-top: 10px; font-weight: 600; }
11         input { width: 100%; max-width: 360px; padding: 6px; box-sizing: border-b
   ox; }
12         button { margin-top: 16px; padding: 8px 20px; }
13     </style>
14 </head>
15
16 <body>
17     <!-- aquí se monta el componente raíz que se declara en MainWindow.xaml -->
18     <div id="app">Cargando...</div>
19
20     <script src="_framework/blazor.webview.js"></script>
21 </body>
22
23 </html>
```

Qué está pasando:
Línea 7: `<base href="/" />` indica la raíz desde la que se resuelven las direcciones relativas. Blazor la necesita.

Línea 18: el `<div id="app">` es el **lugar reservado**. En la Parte D le decís a `BlazorWebView` que el componente `CreateCustomer` se muestre dentro del elemento con ese `id`. El texto "Cargando..." se ve un instante y después lo reemplaza el componente.

Línea 20: el script `blazor.webview.js` es el que conecta la página con el código C#. Ese archivo no existe en tu carpeta: lo aporta el paquete del `BlazorWebView` (por eso hizo falta el SDK Razor). Sin esta línea, el formulario se ve pero los botones no hacen nada.

El bloque `<style>` solo le da un aspecto prolijo al formulario (que en el Módulo 7 escribiste sin estilos). No cambia el comportamiento.

Parte C — Registrar los servicios del BlazorWebView

Ubicación

`src/Desktop/CompositionRoot.cs` (archivo existente, se agrega una línea)

Descripción

El `BlazorWebView` necesita algunos servicios propios para funcionar. Se registran con un método que trae el paquete, `AddWpfBlazorWebView()`, en el composition root, junto a los demás registros. Agregá las tres líneas marcadas antes de los "servicios propios del host":

```
using Microsoft.Extensions.Configuration;
using Microsoft.Extensions.DependencyInjection;

namespace CommerX.Desktop;

// composition root — el único lugar de la aplicación que conoce todas las capas y arma las dependencias
public static class CompositionRoot
{
    public static IServiceCollection AddCommerX(
        this IServiceCollection services,
        IConfiguration configuration)
    {
        // la cadena de conexión viene de appsettings.json; si falta, se falla al arrancar con un mensaje claro
        var connectionString = configuration.GetConnectionString("CommerX")
            ?? throw new InvalidOperationException(
                "Falta la cadena de conexión 'CommerX' en appsettings.json (sección ConnectionStrings).");

        // Application: casos de uso y validación
        services.AddCustomerUseCases();

        // InterfaceAdapter: presenters y view models
        services.AddCustomerPresenter();
        services.AddUpdateCustomerPresenter();
        services.AddCustomerViewModel();

        // Infrastructure: DbContext y repositorios
        services.AddInfrastructure(connectionString);

```

```
29          // servicios que necesita el BlazorWebView para mostrar componentes Razor
30          services.AddWpfBlazorWebView();
31
32          // servicios propios del host
33          services.AddSingleton<DatabaseHealthCheck>();
34          services.AddSingleton<MainWindow>();
35
36          return services;
37      }
38  }
```

Qué está pasando:
Línea 30: registra los servicios internos del control. No cambia ni reemplaza nada de lo que ya tenías: el composition root sigue siendo el único lugar donde se arma el contenedor, y ahora tiene un registro más.

Línea 33: los servicios del host que ya existían. Fijate que **no se registra** `CreateCustomer` acá: los componentes Razor no se registran, los crea Blazor. Lo que sí está registrado, desde el Módulo 7, es su `CreateCustomerViewModel`.

> **Comprobá:** volvé a correr las pruebas del composition root que hiciste en el Módulo 9 con `dotnet test tests/CommerX.Desktop.Tests`. Tienen que seguir en verde: el registro nuevo no rompe la composición.

Parte D — Mostrar `CreateCustomer` en la ventana

Ubicación

`src/Desktop/MainWindow.xaml` y `src/Desktop/MainWindow.xaml.cs` (archivos existentes, se reemplazan)

Descripción

Es la parte central del módulo. La ventana pasa a tener dos zonas: arriba, el estado de la base de datos (lo del Módulo 9) y abajo, el `BlazorWebView` con el formulario. Reemplazá el contenido de `MainWindow.xaml`:

```
1  <Window x:Class="CommerX.Desktop.MainWindow"
2          xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
```

```
3         xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
4         xmlns:blazor="clr-namespace:Microsoft.AspNetCore.Components.WebView.Wpf;a
ssembly=Microsoft.AspNetCore.Components.WebView.Wpf"
5         xmlns:views="clr-namespace:CommerX.InterfaceAdapter.Views.Customers;assem
bly=CommerX.InterfaceAdapter.Views"
6         Title="CommerX" Height="760" Width="620"
7         WindowStartupLocation="CenterScreen"
8         Loaded="OnLoaded">
9     <DockPanel>
10        <StackPanel DockPanel.Dock="Top" Margin="24,16,24,0">
11            <TextBlock Text="CommerX" FontSize="28" FontWeight="Bold" />
12            <StackPanel Orientation="Horizontal" Margin="0,8,0,0">
13                <TextBlock x:Name="StatusText" Text="Base de datos: sin comproba
r" FontSize="14" VerticalAlignment="Center" />
14                <Button Content="Comprobar conexión" Click="OnCheckConnection" Pa
dding="12,4" Margin="16,0,0,0" />
15            </StackPanel>
16        </StackPanel>
17
18        <!-- la pantalla de alta es un componente Razor alojado en un WebView2 de
ntro de la ventana WPF -->
19        <blazor:BlazorWebView x:Name="BlazorView" HostPage="wwwroot\index.html">
20            <blazor:BlazorWebView.RootComponents>
21                <blazor:RootComponent Selector="#app" ComponentType="{x:Type view
s:CreateCustomer}" />
22            </blazor:BlazorWebView.RootComponents>
23        </blazor:BlazorWebView>
24    </DockPanel>
25</Window>
```

Qué está pasando:
Líneas 4 y 5: dos *alias* de espacio de nombres, para poder escribir `blazor:` y `views:` en el XAML. El primero apunta al paquete del `BlazorWebView` y el segundo al proyecto de Views.

Línea 19: el control. `HostPage` es la página de la Parte B, con la ruta relativa a la carpeta de salida.

Línea 21: el **componente raíz**: `Selector="#app"` es el lugar reservado del `index.html` y `ComponentType` es el componente que se muestra ahí, en este caso el `CreateCustomer` que escribiste en el Módulo 7.

Ahora el archivo pareja, `MainWindow.xaml.cs`. Se agrega un parámetro al constructor y una línea:

```
1  using System.Windows;
2
3
4  namespace CommerX.Desktop;
5
6  public partial class MainWindow : Window
7  {
8      private readonly DatabaseHealthCheck _healthCheck;
9
10     // el contenedor inyecta la dependencia: la ventana no sabe cómo se comprueba
   la base
11     public MainWindow(DatabaseHealthCheck healthCheck, IServiceProvider services)
12     {
13         _healthCheck = healthCheck;
14         InitializeComponent();
15
16         // los componentes Razor resuelven sus dependencias del mismo contenedor
   que armó el composition root
17         BlazorView.Services = services;
18     }
19
20     private async void OnLoaded(object sender, RoutedEventArgs e) => await CheckA
   sync();
21
22     private async void OnCheckConnection(object sender, RoutedEventArgs e) => awa
   it CheckAsync();
23
24     private async Task CheckAsync()
25     {
26         StatusText.Text = "Base de datos: comprobando...";
27
28         var connected = await _healthCheck.CanConnectAsync();
29
30         StatusText.Text = connected
31             ? "Base de datos: conectada"
32             : "Base de datos: sin conexión";
33     }
34 }
```

Qué está pasando:

Línea 11: el constructor pide un `IServiceProvider`, que es el contenedor de dependencias completo. Como la ventana se resuelve del contenedor (Módulo 9), el contenedor mismo se lo da.

Línea 17: le entrega ese contenedor al `BlazorWebView`. **Esta es la línea que conecta las dos mitades:** gracias a ella, cuando el componente `CreateCustom`

`er` pide su `CreateCustomerViewModel` (por `OwningComponentBase`, Módulo 7), lo recibe del mismo contenedor que armó el composition root, con el caso de uso, el presenter y el repositorio ya conectados. Sin esta línea, el componente no encuentra sus dependencias.

> **No toques `App.xaml.cs`:** el host genérico y la ventana se siguen creando como en el Módulo 9. En este módulo no cambia ninguna otra clase de C#.

Parte E — Ejecutar y registrar un cliente

Descripción

Con todo conectado, ejecutá el host desde la carpeta raíz de la solución (o con **F5** en Visual Studio):

```
dotnet run --project src/Desktop/CommerX.Desktop.csproj
```

> **Resultado esperado:** se abre la ventana **CommerX** con el texto "Base de datos: conectada" arriba y, debajo, el formulario **"CU-CLI-001 — Alta de Cliente"** con sus siete campos y el botón *Guardar*.

Prueba 1 — Un cliente válido

Completá el formulario con datos reales (por ejemplo: *Ana*, *Pérez*, documento *30123456*, un email con formato válido, un teléfono, un domicilio y una fecha de nacimiento) y tocá **Guardar**. Debajo del botón tiene que aparecer: *"Cliente 'Ana Pérez' registrado correctamente."*

Prueba 2 — El mismo documento otra vez

Tocá **Guardar** de nuevo sin cambiar nada. Ahora tiene que aparecer *"El documento 30123456 ya está registrado."* Esa regla (documento único) la aplica el caso de uso al consultar la base de datos, y la ves reflejada en la pantalla sin que la View sepa nada de ella.

Prueba 3 — Datos inválidos

Dejá el nombre vacío, escribí un documento muy corto (por ejemplo, *x*) y un email sin formato (por ejemplo, *noesmail*) y tocá **Guardar**. Tienen que aparecer los mensajes de los Guards (Módulo 5), que reúne el Validator del Módulo 6, separados por punto y coma (*"El campo 'FirstName' es requerido.; …"*).

Comprobar en la base de datos

Para ver que el cliente quedó guardado de verdad, en Visual Studio abrí el **Explorador de objetos de SQL Server** (menú **Ver**) → `(localdb)\MSSQLLocalDB` → Bases de datos → `CommerX` → Tablas → clic derecho sobre `dbo.Customers` → **Ver datos**. Tiene que haber una fila con el cliente de la Prueba 1.

> **Si algo no funciona:**
> **La ventana abre, pero la zona del formulario queda en blanco y no aparece ningún error:** falta la línea `BlazorView.Services = services;` de la Parte D (o no está instalado el runtime de WebView2).
> **Se ve el texto "Cargando..." y no avanza:** falta el script `blazor.webview.js` de la Parte B.
> **El programa se cierra al abrir con el mensaje "Unable to find the required services. Please add all the required services by calling 'IServiceCollection.AddWpfBlazorWebView'":** falta el registro de la Parte C.

Checklist de verificación

Archivo / Tarea	Verificación
`CommerX.Desktop.csproj`	SDK `Microsoft.NET.Sdk.Razor`; referencia a `CommerX.InterfaceAdapter.Views`; paquete `Microsoft.AspNetCore.Components.WebView.Wpf` 9.0.120; compila con 0 errores y 0 advertencias
`wwwroot/index.html`	Tiene `<base href="/" />`, un elemento con `id="app"` y el script `_framework/blazor.webview.js`
`CompositionRoot.cs`	Llama a `services.AddWpfBlazorWebView()`; no registra el componente `CreateCustomer`

Archivo / Tarea	Verificación
`MainWindow.xaml`	Tiene un `BlazorWebView` con `HostPage="wwwroot\index.html"` y un `RootComponent` con `Selector="#app"` y `ComponentType` `CreateCustomer`
`MainWindow.xaml.cs`	El constructor recibe `IServiceProvider` y asigna `BlazorView.Services` después de `InitializeComponent()`
Ejecución	Se muestra el formulario; un cliente válido se guarda; el mismo documento es rechazado; los datos inválidos muestran los mensajes de validación; la fila aparece en `dbo.Customers`
Capas interiores	No se modificó ningún archivo de Domain, Application, InterfaceAdapter ni de la View

Preguntas de reflexión

1. **En esta actividad no modificaste ninguna capa interior. ¿Qué te dice eso sobre la arquitectura?**
   Pista: pensá en lo que habría que cambiar si la pantalla fuera una página web en lugar de un control dentro de una ventana de escritorio.
2. **¿Por qué `CreateCustomer` no se registra en el contenedor de dependencias, pero `CreateCustomerViewModel` sí?**
   Pista: ¿quién crea un componente Razor? ¿Y quién crea un ViewModel?
3. **¿Qué hace la línea `BlazorView.Services = services;` y qué pasaría si la quitaras?**
   Pista: el componente pide un ViewModel que necesita un caso de uso. ¿De dónde lo saca?
4. **La regla "el documento ya está registrado" aparece en pantalla. ¿En qué capa vive esa regla y por qué la View no la conoce?**
   Pista: seguí el camino del botón *Guardar* hasta el repositorio y de vuelta hasta el mensaje.
5. **¿Por qué este módulo es una modificación de la capa más externa y no de las interiores?**
   Pista: recordá hacia dónde apuntan las dependencias y qué proyecto

conoce a todos los demás.

6. **Si un compañero quisiera mostrar una segunda pantalla (por ejemplo, para actualizar un cliente), ¿qué archivos del host tendría que tocar?** *Pista:* ¿hay que registrar algo nuevo en el composition root? ¿qué cambia en el `RootComponent`?